

Tool Calling, Agent Loops, and Routing Notes

Tool schemas, validation, observations, multi-step actions, iteration limits, routing, and tool errors.

Engineering notes and worked references

Contents

1. What tool calling means
2. Tool schemas
3. Argument validation
4. Tool results
5. Multi-step tool use
6. Iteration limits
7. Repeated tool loops
8. Routing
9. Tool errors
10. Direct response versus tool use

1. What tool calling means

A tool-capable model can return a structured request that names a function and supplies arguments. The application, not the model, executes the actual function.

This separation is important. The model proposes an action. Application code validates it, performs it, and returns the result as another message or observation.

Example from use

The model can request `save_note(title='Top-k', content='...')`.

A calculator tool should run normal code, not ask the model to mentally simulate the result.

Things people mix up

- Saying the model 'called the database' when your Python code did it.
- Executing arbitrary function names.
- Skipping argument validation.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Application boundary

In what tool calling means, the model is never the final authority on whether an action happened. The Python function or database operation is the source of truth. The model can ask, but the application validates and executes.

This becomes especially important when the action changes saved state. A tool result should contain the real note ID, real database result, or real error produced by the function. The assistant response comes after that observation.

What to log

- Tool name.
- Validated arguments or a safe summary of them.
- Step number.
- Result status and returned identifier.

These logs are enough to explain most tool-loop bugs. There is no need to expose hidden model reasoning. The observable action sequence tells you whether the router, schema, validation, or function behaved incorrectly.

2. Tool schemas

A tool description tells the model the function name, what it does, and which arguments it accepts.

Good schemas are specific enough that two tools do not sound interchangeable. Argument types should match the actual Python function.

Example from use

`find_notes(query: string, top_k: integer)` is clearer than `do_search(data)`.

`save_note` can require title and content while `session_id` is supplied by trusted application state.

Things people mix up

- Writing vague tool descriptions.
- Letting the model supply a user ID that should come from the server session.
- Changing the function signature without updating the schema.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Application boundary

In tool schemas, the model is never the final authority on whether an action happened. The Python function or database operation is the source of truth. The model can ask, but the application validates and executes.

This becomes especially important when the action changes saved state. A tool result should contain the real note ID, real database result, or real error produced by the function. The assistant response comes after that observation.

What to log

- Tool name.
- Validated arguments or a safe summary of them.
- Step number.
- Result status and returned identifier.

These logs are enough to explain most tool-loop bugs. There is no need to expose hidden model reasoning. The observable action sequence tells you whether the router, schema, validation, or function behaved incorrectly.

3. Argument validation

Tool arguments are model-generated input and should be validated before execution. Check types, allowed ranges, identifiers, and required fields.

Validation prevents small formatting mistakes from turning into database or runtime errors and provides a clean place to reject unsafe requests.

Example from use

Clamp or reject top_k outside an allowed range.

Reject a note ID that does not belong to the current session.

Things people mix up

- Passing raw arguments directly into SQL.
- Converting every invalid value silently and hiding the bug.
- Returning a Python traceback to the user.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?

- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Application boundary

In argument validation, the model is never the final authority on whether an action happened. The Python function or database operation is the source of truth. The model can ask, but the application validates and executes.

This becomes especially important when the action changes saved state. A tool result should contain the real note ID, real database result, or real error produced by the function. The assistant response comes after that observation.

What to log

- Tool name.
- Validated arguments or a safe summary of them.
- Step number.
- Result status and returned identifier.

These logs are enough to explain most tool-loop bugs. There is no need to expose hidden model reasoning. The observable action sequence tells you whether the router, schema, validation, or function behaved incorrectly.

4. Tool results

After a tool runs, return a compact result that contains what the model needs for the next step. Do not dump unnecessary internal state.

The tool result should be truthful even when the function failed. A failed search should say no results or return an error object rather than a made-up success message.

Example from use

find_notes can return note IDs, titles, and short snippets.

save_note can return note_id and saved_at.

Things people mix up

- Returning huge database rows when only three fields are needed.
- Letting the model invent a tool result.
- Hiding errors by returning an empty success object.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Application boundary

In tool results, the model is never the final authority on whether an action happened. The Python function or database operation is the source of truth. The model can ask, but the application validates and executes.

This becomes especially important when the action changes saved state. A tool result should contain the real note ID, real database result, or real error produced by the function. The assistant response comes after that observation.

What to log

- Tool name.
- Validated arguments or a safe summary of them.
- Step number.
- Result status and returned identifier.

These logs are enough to explain most tool-loop bugs. There is no need to expose hidden model reasoning. The observable action sequence tells you whether the router, schema, validation, or function behaved incorrectly.

5. Multi-step tool use

Some requests need more than one action. A model might search notes, compare two retrieved passages, save the explanation, then return the note ID.

Each step should have a clear reason and the application should keep a maximum step count.

Example from use

Question: 'Compare Chroma and FAISS and save this.' Search source sections, compose comparison, then save note.

Question requiring a documented fee plus arithmetic can retrieve the fee then call a calculator.

Things people mix up

- Forcing every request through several tools.
- Allowing an unbounded loop.
- Running a save action before the content has been validated.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Application boundary

In multi-step tool use, the model is never the final authority on whether an action happened. The Python function or database operation is the source of truth. The model can ask, but the application validates and executes.

This becomes especially important when the action changes saved state. A tool result should contain the real note ID, real database result, or real error produced by the function. The assistant response comes after that observation.

What to log

- Tool name.
- Validated arguments or a safe summary of them.
- Step number.
- Result status and returned identifier.

These logs are enough to explain most tool-loop bugs. There is no need to expose hidden model reasoning. The observable action sequence tells you whether the router, schema, validation, or function behaved incorrectly.

6. Iteration limits

A tool loop should stop after a sensible number of steps. The exact number depends on the task, but a small project rarely needs dozens of calls.

A limit protects against repeated searches, bad tool schemas, and a model that keeps asking for the same action.

Example from use

Stop after 5 or 6 tool steps and return a controlled error if no final answer is produced.

Record tool name and step number for debugging.

Things people mix up

- Using while True with no guard.
- Silently discarding the last tool error.
- Setting the limit so low that normal two-step tasks cannot finish.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Application boundary

In iteration limits, the model is never the final authority on whether an action happened. The Python function or database operation is the source of truth. The model can ask, but the application validates and executes.

This becomes especially important when the action changes saved state. A tool result should contain the real note ID, real database result, or real error produced by the function. The assistant response comes after that observation.

What to log

- Tool name.
- Validated arguments or a safe summary of them.
- Step number.
- Result status and returned identifier.

These logs are enough to explain most tool-loop bugs. There is no need to expose hidden model reasoning. The observable action sequence tells you whether the router, schema, validation, or function behaved incorrectly.

7. Repeated tool loops

A common failure is repeated use of the same tool with the same arguments even though the result is already available.

Detect or log repeated calls. The model may need a different tool, a better prompt, or a clearer observation.

Example from use

If `document_search('top-k')` returns the passage, calling the same search four more times is probably not useful.

If a calculator returns 56, another identical calculator call should not be needed.

Things people mix up

- Assuming repeated calls mean the tool itself is broken.
- Ignoring duplicate requests because they are cheap in development.
- Increasing the max loop count instead of finding the cause.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Application boundary

In repeated tool loops, the model is never the final authority on whether an action happened. The Python function or database operation is the source of truth. The model can ask, but the application validates and executes.

This becomes especially important when the action changes saved state. A tool result should contain the real note ID, real database result, or real error produced by the function. The assistant response comes after that observation.

What to log

- Tool name.
- Validated arguments or a safe summary of them.
- Step number.
- Result status and returned identifier.

These logs are enough to explain most tool-loop bugs. There is no need to expose hidden model reasoning. The observable action sequence tells you whether the router, schema, validation, or function behaved incorrectly.

8. Routing

A router chooses which path should handle the request. A small assistant may route to document search, a direct conversational response, or a tool action.

Routing can be rule-based, model-based, or a mix. The route should reflect intent, not superficial features like whether the message contains a number.

Example from use

'Hello' can use a direct path with no retrieval.

'What is top-k?' should search the notes.

'Save that explanation' should use the saved-note action.

Things people mix up

- Running vector search for greetings.
- Routing every question containing digits to a calculator.
- Hiding the route decision during debugging.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Application boundary

In routing, the model is never the final authority on whether an action happened. The Python function or database operation is the source of truth. The model can ask, but the application validates and executes.

This becomes especially important when the action changes saved state. A tool result should contain the real note ID, real database result, or real error produced by the function. The assistant response comes after that observation.

What to log

- Tool name.
- Validated arguments or a safe summary of them.
- Step number.
- Result status and returned identifier.

These logs are enough to explain most tool-loop bugs. There is no need to expose hidden model reasoning. The observable action sequence tells you whether the router, schema, validation, or function behaved incorrectly.

9. Tool errors

Tool functions can fail because of missing records, invalid arguments, database errors, or external service problems.

Return a structured error that the model can explain or recover from. Do not fabricate a successful observation to keep the conversation moving.

Example from use

load_note with an unknown ID can return not_found.

A database exception can produce a generic temporary failure while the server logs the full detail.

Things people mix up

- Passing stack traces into the model and user output.
- Retrying a deterministic not-found error.
- Letting the model claim an action succeeded after the tool reported failure.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Application boundary

In tool errors, the model is never the final authority on whether an action happened. The Python function or database operation is the source of truth. The model can ask, but the application validates and executes.

This becomes especially important when the action changes saved state. A tool result should contain the real note ID, real database result, or real error produced by the function. The assistant response comes after that observation.

What to log

- Tool name.
- Validated arguments or a safe summary of them.
- Step number.
- Result status and returned identifier.

These logs are enough to explain most tool-loop bugs. There is no need to expose hidden model reasoning. The observable action sequence tells you whether the router, schema, validation, or function behaved incorrectly.

10. Direct response versus tool use

A tool should exist because it gives the application a capability the model does not have directly, such as reading stored state, running a search, saving data, or doing exact arithmetic.

Do not use tools for ordinary conversational phrasing. The model can answer a greeting without calling a database or vector store.

Example from use

'Thanks' needs no tool.

'List my saved notes' needs a database-backed action.

Things people mix up

- Adding a tool only to make the architecture look more agentic.
- Calling search for every user turn.
- Using the LLM as a substitute for a deterministic lookup.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Application boundary

In direct response versus tool use, the model is never the final authority on whether an action happened. The Python function or database operation is the source of truth. The model can ask, but the application validates and executes.

This becomes especially important when the action changes saved state. A tool result should contain the real note ID, real database result, or real error produced by the function. The assistant response comes after that observation.

What to log

- Tool name.
- Validated arguments or a safe summary of them.
- Step number.
- Result status and returned identifier.

These logs are enough to explain most tool-loop bugs. There is no need to expose hidden model reasoning. The observable action sequence tells you whether the router, schema, validation, or function behaved incorrectly.